

Segment 3 — Detrending & Bandpass Filtering

Spandan: Contactless Vital Sign Monitoring — EEE 312 DSP Project, BUET

Team Spandan

August 5, 2026

Why This Segment Exists

Segment 2 gave us three raw signals — $R(t)$, $G(t)$, $B(t)$ — one spatially-averaged pixel value per video frame, per color channel. If you plot one of these raw traces, it does **not** look like a heartbeat. It looks like a slowly wandering curve with a faint, fast ripple riding on top of it, half-buried in noise. Segment 3's whole job is classical DSP housekeeping: strip away everything that isn't the 0.7-4 Hz pulse band, so that Segment 4 (CHROM/POS combination) and Segment 5 (FFT heart rate, SpO2) have something clean to actually work with.

Nothing in this segment does anything clever or algorithm-specific to rPPG. It's two textbook DSP operations — detrending and bandpass filtering — applied carefully, with parameters that are justified rather than guessed.

Intuition First

Intuition first. Imagine trying to hear a metronome ticking softly in a room where someone is also slowly turning a dimmer switch up and down, and there's a low hum from an air conditioner in the background. The metronome tick is the pulse signal. The dimmer switch is the slow illumination drift — it changes the *overall loudness* of everything over many seconds, but it isn't the tick. The AC hum is out-of-band noise, sensor noise, tiny motion jitter — real, but not the tick either. Segment 3 is the pair of noise-cancelling headphones tuned specifically to let the metronome's tempo through and cut everything above and below it.

Why Raw R/G/B Has Drift and Out-of-Band Noise in the First Place

The raw traces coming out of Segment 2 carry several things mixed together, at very different timescales:

1. **The pulse itself** — blood volume under the forehead skin rising and falling with each heartbeat, at roughly 0.7-4 Hz (42-240 bpm). This is the signal of interest, and it is a genuinely small fraction of the total brightness variation.
2. **Slow illumination drift** — ambient light changing over many seconds (a cloud, a flickering fixture, the camera's auto-exposure settling in), or the subject's overall skin tone/position shifting slightly. This happens on the order of the whole recording's length, far below 0.7 Hz.

3. **High-frequency noise** — camera sensor noise, small unavoidable motion jitter, anything faster than a heart physically can beat. This sits above 4 Hz.

A raw trace is the sum of all three. Detrending removes (2). Bandpass filtering removes both (2) and (3) more precisely, keeping only the band where (1) actually lives.

Common Beginner Mistake

Common beginner mistake. It's tempting to assume the raw signal *is* basically the pulse already, just “a bit noisy,” and to skip straight to an FFT or a peak-counting algorithm. In practice the drift component is usually much larger in amplitude than the pulse ripple itself — an FFT run on the raw, undetrended signal will often show almost all its energy crammed into a huge near-0 Hz peak from the drift, with the actual pulse peak barely visible by comparison. Detrending and bandpass filtering aren't optional cleanup steps here — without them, later frequency-domain analysis is looking at the wrong thing entirely.

Polynomial Detrending, Intuition

`filtering/detrendSignal.m` fits a smooth polynomial curve to the whole raw signal and subtracts it, using MATLAB's built-in `detrend(x, n)`, where `n` is the polynomial's degree.

Intuition First

Intuition first. Think of detrending as asking: “if I ignore the fast wiggles entirely, what's the big, slow-moving shape this signal is following?” A least-squares polynomial fit answers exactly that — it finds the smooth curve of degree `n` that best follows the overall shape of the data, ignoring the fast stuff (which averages out across the fit because it wiggles too quickly to influence a smooth low-degree curve). Subtracting that curve leaves behind whatever the fit *couldn't* explain — which, if the degree is chosen well, is mostly the fast pulse ripple plus noise.

This project uses a **cubic (3rd-order) polynomial**. The reasoning:

- Order 0 or 1 (mean removal, or mean + straight-line removal) can't bend to follow a curved drift — real illumination drift over a 60-90 second recording usually isn't perfectly linear.
- Order 3 gives enough flexibility to follow a smooth, gently curving drift without having enough wiggle room to start tracking the pulse ripple itself.
- The frequency argument makes this precise: a degree-3 polynomial fit to an 80-second recording has almost all its energy at frequencies around $1/T \approx 0.01\text{-}0.04$ Hz — more than an order of magnitude below the 0.7 Hz pulse band floor. It physically cannot oscillate fast enough to remove real pulse content.

Pause and Check Yourself

Pause and check yourself. If someone suggested using a degree-15 polynomial for detrending “to get an even better fit,” what would actually go wrong with the output signal, and why?

Butterworth Bandpass Intuition — Why 0.7-4 Hz Maps to 40-240 bpm

A heart rate in beats per minute converts to a frequency in Hz by dividing by 60 (60 seconds per minute): 40 bpm is $40/60 \approx 0.67$ Hz, and 240 bpm is $240/60 = 4$ Hz. This project rounds the lower edge up slightly to 0.7 Hz as a small safety margin (filters don't cut off perfectly sharply, and resting heart rates in this dataset never come close to 40 bpm anyway), giving the working band **0.7-4 Hz**.

`filtering/bandpassClean.m` designs a Butterworth bandpass filter over exactly that range using `butter()`, then applies it. A Butterworth filter is chosen specifically because it has a **maximally flat passband** — within the band you want to keep, it distorts the relative amplitude of different frequencies as little as possible, compared to filter families (like Chebyshev) that trade passband flatness for a steeper cutoff. Since later stages care about waveform shape, not just “is there energy somewhere in this band,” passband flatness matters more here than an aggressively steep rolloff would.

Intuition First

Intuition first. A low-order Butterworth bandpass is like a gentle bouncer at a club door checking IDs at a glance — fast, doesn't hassle anyone who's clearly fine, lets borderline cases through without overthinking it. A very high-order filter is like a bouncer who pat-searches everyone at length — technically more thorough at rejecting anyone outside the age range, but slows down and distorts the experience of everyone passing through, including the people who were supposed to get in cleanly.

Why a low order (2, giving an effective 4th-order bandpass). `butter(N, ..., 'bandpass')` internally doubles the order — asking for $N = 2$ produces a filter with the equivalent of 4 poles. Going higher (order 4, 6...) would reject out-of-band content more aggressively, but at the cost of more ringing and overshoot near sharp features in the passband, and more numerical sensitivity for a narrow band like this one relative to the ~ 15 Hz Nyquist frequency of a ~ 29 -30 fps video. A low order that does a clean job is the right trade-off, not a shortcut.

filtfilt vs filter — Why Phase Distortion Specifically Matters Here

`butter()` only produces filter *coefficients* — numbers describing the filter, not an action. Applying them is a separate step, and MATLAB offers two very different ways to do it:

- `filter(b, a, x)` applies the filter once, moving forward through the signal. Every real digital filter delays different frequencies by different amounts as a side effect — this is called phase distortion, and a single `filter()` pass always has it.
- `filtfilt(b, a, x)` applies the same filter twice: once forward, then again backward over the result. Running the filter backward exactly reverses the timing distortion the forward pass introduced — a delay followed by an equal-and-opposite “undelay” cancels to zero net phase shift. What's left is a pure magnitude-only filtering effect, perfectly time-aligned to the original signal.

Common Beginner Mistake

Common beginner mistake. For FFT-based heart rate alone, a plain `filter()` call would honestly work fine — a phase shift doesn’t change *which* frequency has the most energy, only *when* in time that energy appears, and an FFT throws away timing information anyway. It’s easy to conclude from that fact that `filter()` is good enough everywhere in this project. It is not. This project’s SpO2 stage needs the AC (pulsatile) *amplitude* of the waveform in each color channel, compared against each other with correct relative timing. A phase-shifted signal doesn’t just look “shifted” — because different channels or different filter implementations can shift by slightly different amounts, using `filter()` risks quietly distorting exactly the waveform-shape information the SpO2 ratio-of-ratios calculation depends on later. Using `filtfilt()` everywhere in this segment, even in the places where it isn’t strictly required yet, means Segment 3’s output is safe to feed into *every* downstream stage without having to remember which ones can tolerate a phase shift and which can’t.

Pause and Check Yourself

Pause and check yourself. If `filtfilt()` runs the filter twice (forward, then backward), why doesn’t that make the filter’s effective order twice as aggressive as intended? (Hint: think about what “twice as aggressive” would actually mean for the rolloff steepness versus what running backward specifically undoes.)

Why the Cutoffs Must Be Computed From the Subject’s Own `fs`

`butter()` doesn’t take cutoff frequencies in Hz — it wants them normalized to the Nyquist frequency (half the sampling rate), as a fraction between 0 and 1. `bandpassClean.m` computes this fresh, every call, from the `frameRate` argument it’s given:

```
nyquistHz = frameRate / 2;
lowCutoffNormalized = lowCutoffHz / nyquistHz;
highCutoffNormalized = highCutoffHz / nyquistHz;
```

`docs/DATA_FORMAT.md` confirms UBFC-rPPG videos run anywhere from about 28.6 to 29.8 fps, differing per subject — not a clean, uniform 30 fps. If this function hardcoded a Nyquist frequency based on an assumed 30 fps, the normalized cutoffs would put the *actual* filtered band at a very slightly different place in Hz than 0.7-4 Hz for every subject whose true `fs` isn’t exactly 30. Critically, this would produce **no error and no warning** — the filter still runs, still returns a plausible-looking output, it’s just filtering marginally the wrong band. That’s precisely the kind of small, silent, believable-looking bug that undermines an otherwise-correct DSP pipeline. Reading `frameRate` fresh from the argument (which itself traces back to `VideoReader.FrameRate`, read directly from each subject’s own video file) closes that gap completely, for free.

Common Pitfalls in This Segment

Over-filtering flattens real pulse features. Both the detrend order and the filter order/bandwidth choices in this segment were deliberately kept conservative. It’s tempting to think “more aggressive filtering = cleaner signal,” but every filter operation risks removing not just noise but also genuine variation in pulse *shape* (dicotic notch shape, amplitude modulation from breathing, etc.) that later stages — especially SpO2’s AC amplitude extraction — may actually need.

Filtering exactly as much as is justified, and no more, is the goal, not filtering as aggressively as possible.

Using a global `fs` instead of the subject’s actual `fs`. Covered in detail above — worth repeating as its own pitfall because it’s the single easiest mistake to make silently. Always pass each subject’s own `fs` (loaded from their `_rgb_traces.mat` file, which itself came from their own video’s `VideoReader.FrameRate`) into `bandpassClean.m`. Never substitute a constant, even “just for a quick test.”

Common Beginner Mistake

Common beginner mistake. Trusting a filtered `.mat` file just because the code ran without an error. A bandpass filter given the wrong cutoffs, or a detrend order high enough to eat into the pulse band, will still produce a `1 x N` output vector with no complaint — it just won’t be the signal you think it is. This is exactly why this segment mandates a sanity-check PNG for every subject, showing raw, detrended, and filtered versions of the same channel stacked on the same time axis: a human can tell at a glance whether the drift is actually gone and whether what’s left looks like a plausible repeating pulse waveform, in a way that “the script didn’t crash” never can.

How This Segment Feeds Into Segment 4

The output of this segment — `data/processed/<subjectID>_filtered_traces.mat`, containing `R_filtered`, `G_filtered`, `B_filtered`, `fs`, `subjectID`, `detrendOrder`, and `filterOrder` — is the entire input Segment 4 needs. Segment 4 (`pulseextraction/chromCombine.m` and `pulseextraction/posCombine.m`) takes these three cleaned, bandpassed channels and combines them using the published CHROM and POS algorithms into a single pulse signal — a motion-robust combination that does meaningfully better than just picking the green channel alone, which is what a naive single-channel approach would do.

Everything downstream of Segment 3 assumes its input channels are already detrended, bandpassed, and phase-preserved. That assumption is exactly what this segment is responsible for delivering. If a subject’s sanity PNG still shows visible drift, or the filtered plot looks flat or still noisy rather than like a repeating wave, that is the moment to investigate — before, not after, that subject’s data is combined and analyzed three more stages down the line.